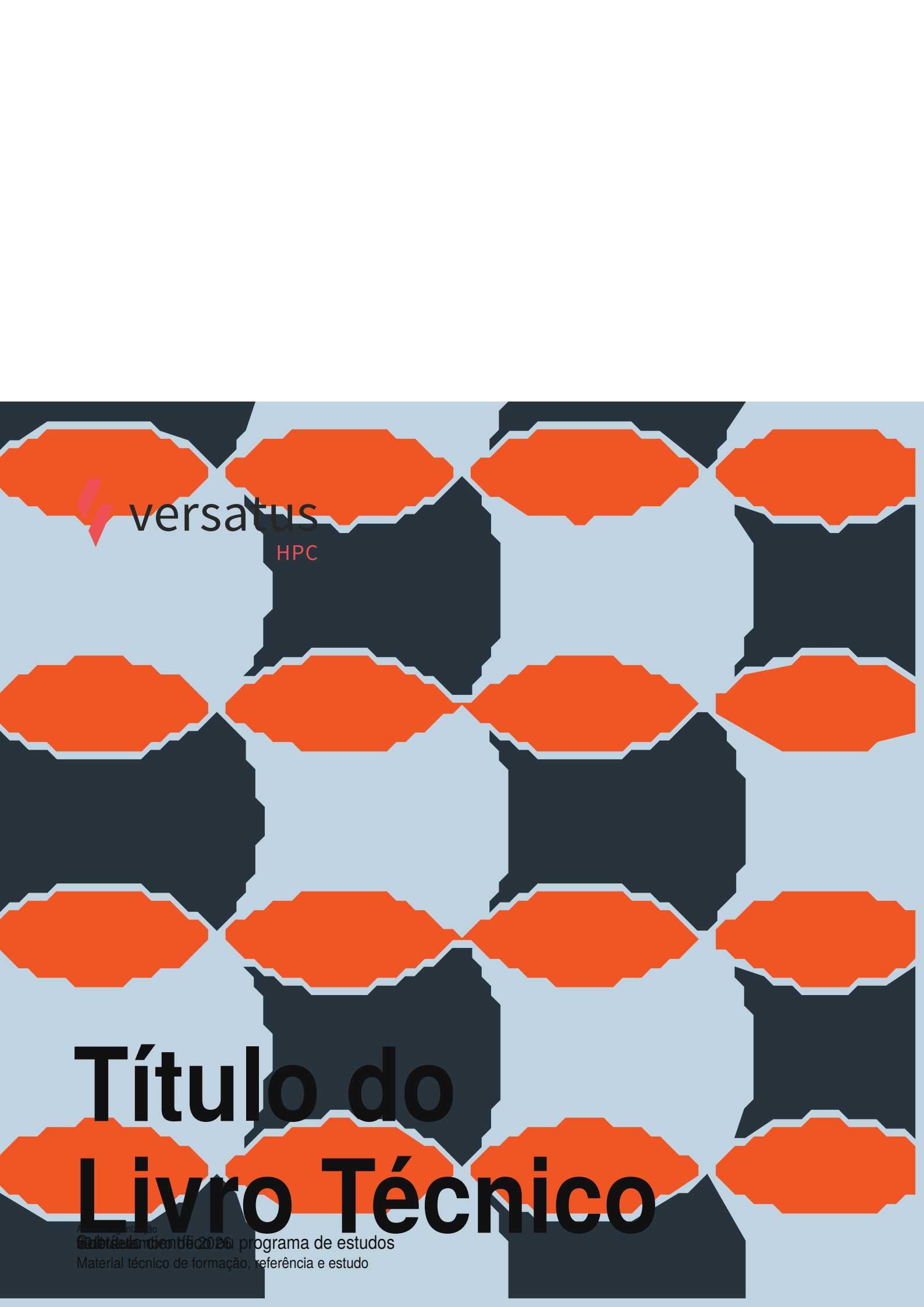




versatus

HPC

Título do Livro Técnico

Atualização
Substituição 2026 programa de estudos
Material técnico de formação, referência e estudo

Sumário

1 Estruturas de Texto e Tabelas	1
1.1 Três parágrafos contínuos	1
1.2 Tabela pequena	1
1.3 Tabela média	2
1.4 Tabela grande	2
2 Equações e Modelos Matemáticos	4
2.1 Equação pequena	4
2.2 Equação média	4
2.3 Equação grande em múltiplas linhas	5
2.4 Bloco técnico com equação	5
3 Figuras, Gráficos e Blocos Multicoluna	6
3.1 Um gráfico	6
3.2 Dois gráficos lado a lado	7
3.3 Quatro gráficos em matriz 2 x 2	7
3.4 Foto centralizada	8
3.5 Foto com texto ao lado	8
3.6 Dois blocos em duas colunas	8
3.7 Três blocos em três colunas	9
3.8 Quatro blocos em quatro colunas	9
4 Blocos de Código e Terminal	10
4.1 Bloco pequeno de código em terminal	10
4.2 Bloco médio de código em terminal	11
4.3 Bloco grande de código em terminal	12
4.4 Bloco pequeno de código claro	13
4.5 Bloco médio de código claro	13
4.6 Bloco grande de código claro	14
5 Teoremas, Provas e Blocos de Destaque	15
5.1 Enunciação de teorema matemático	15
5.2 Prova de teorema matemático	15
5.3 Enunciação de lema matemático	16
5.4 Enunciação de corolário matemático	16



5.5 Destaque importante de texto	16
5.6 Destaque importante em bloco cinza claro	16
5.7 Destaque importante em bloco colorido	17



Estruturas de Texto e Tabelas

Este capítulo funciona como uma página de referência para o desenho interno de livros técnicos da Versatus HPC. O conteúdo é propositalmente genérico: ele deve ser substituído pelo texto real do projeto, mas preserva exemplos de hierarquia, ritmo editorial, tabelas e parágrafos contínuos. A intenção é facilitar a validação do template antes do início da escrita final.

1.1 Três parágrafos contínuos

A documentação técnica deve privilegiar uma sequência de leitura estável: contexto, decisão, justificativa e consequência. Em materiais didáticos, essa ordem reduz ambiguidade e evita que o leitor precise inferir a arquitetura mental do autor. O texto corrido deve ser usado quando a explicação depende de continuidade conceitual, especialmente em capítulos introdutórios, fundamentos teóricos e discussões de projeto.

Em documentos de engenharia, parágrafos longos demais tendem a prejudicar a manutenção editorial. A recomendação é preservar blocos com uma ideia central por parágrafo, usando listas apenas quando houver enumeração real. Essa disciplina ajuda a converter o mesmo conteúdo em livro, manual, relatório técnico ou white paper sem reescrita estrutural excessiva.

Para programas de estudo, a redação deve separar claramente conceitos estáveis, hipóteses de trabalho, exemplos, restrições e decisões normativas. Essa separação permite que o material seja usado tanto para leitura sequencial quanto para consulta posterior. O template foi desenhado para aceitar ambos os modos sem alterar a identidade visual.

1.2 Tabela pequena

A tabela pequena deve ser usada para parâmetros, convenções, nomenclaturas ou comparações curtas. Ela não deve dominar a página.

Tabela 1.1: Exemplo de tabela pequena

Item	Valor	Observação
CPU	2 sockets	Nós de computação
GPU	8 unidades	Treinamento de modelos
Rede	400 Gb/s	Interconexão de baixa latência

1.3 Tabela média

A tabela média é indicada quando há mais texto por célula. Neste caso, `tabularx` permite que as colunas se ajustem à largura útil da página.

Tabela 1.2: Exemplo de tabela média com largura controlada

Categoria	Uso recomendado	Critério de aceite
Provisionamento	Descrever instalação, imagem base, dependências e sequência de boot.	O leitor consegue reproduzir o ambiente sem decisões implícitas.
Operação	Explicar rotinas administrativas, telemetria e tratamento de falhas.	O procedimento apresenta pré-condições, passos e rollback.
Validação	Registrar testes, métricas, hipóteses e limites conhecidos.	Os resultados são rastreáveis e repetíveis.

1.4 Tabela grande

A tabela grande deve ser usada com moderação. Em livros e manuais, ela é útil para matrizes de requisitos, listas de componentes, parâmetros de configuração e inventários técnicos. Quando a tabela atravessar páginas, use `longtable`.

Tabela 1.3: Exemplo de tabela grande com múltiplas linhas

ID	Componente	Função	Prioridade	Métrica	Evidência	Estado
R01	Kernel	Base de execução do sistema	Alta	Latência e estabilidade	Testes de carga	Aberto
R02	Rede	Comunicação entre nós	Alta	Vazão e perda de pacotes	Benchmarks sintéticos	Aberto
R03	Storage	Persistência para-lela	Alta	IOPS e throughput	IOR, mdtest	Aberto
R04	Scheduler	Alocação de recursos	Alta	Tempo de fila	Simulações de workload	Aberto
R05	Observabilidade	Eventos e métricas	Média	Custo de coleta	Telemetria contínua	Aberto
R06	Segurança	Controle de acesso	Alta	Superfície exposta	Revisão de configuração	Aberto
R07	Documentação	Reprodutibilidade	Média	Cobertura editorial	Checklist de publicação	Aberto

Continua na próxima página

ID	Componente	Função	Prioridade	Métrica	Evidência	Estado
R08	CI/CD	Geração de arte-fatos	Média	Tempo de build	Pipeline automatizado	Aberto
R09	Instalação	Primeira implantação	Alta	Taxa de sucesso	Testes em ambiente limpo	Aberto
R10	Atualização	Evolução controlada	Média	Rollback funcional	Ensaios de verificação	Aberto
R11	Diagnóstico	Suporte operacional	Média	Tempo de triagem	Casos simulados	Aberto
R12	Treinamento	Transferência de conhecimento	Baixa	Conclusão dos módulos	Avaliação interna	Aberto



CAPÍTULO 2

Equações e Modelos Matemáticos

Este capítulo apresenta exemplos de equações pequenas, médias e grandes. A finalidade é validar espaçamento, numeração, quebras de linha, referências cruzadas e integração visual entre texto técnico e formalismo matemático.

2.1 Equação pequena

Equações pequenas podem aparecer em linha, como $E = mc^2$, ou em modo destacado quando forem usadas como referência posterior:

$$T = \frac{D}{B}. \tag{2.1}$$

Na [equation \(2.1\)](#), T representa tempo, D representa volume de dados e B representa largura de banda efetiva.

2.2 Equação média

Equações médias costumam ocupar quase uma linha inteira. Elas devem permanecer legíveis, sem compressão manual excessiva.

$$\mathcal{C}_{\text{total}}(n, m, k) = \alpha n + \beta m \log_2(m) + \gamma k^2 + \delta \sum_{i=1}^n \frac{x_i}{1 + \rho_i}. \tag{2.2}$$

A [equation \(2.2\)](#) ilustra uma expressão composta por termos lineares, logarítmicos, quadráticos e uma soma ponderada. Em documentação técnica, esse formato é adequado para modelos de custo, estimativas de capacidade e hipóteses analíticas.

2.3 Equação grande em múltiplas linhas

Equações grandes devem usar ambientes como `align`, `aligned` ou `multline`. O objetivo é preservar alinhamento sem sacrificar leitura.

$$\mathcal{L}(\theta) = \sum_{b=1}^B \left[\frac{1}{N_b} \sum_{i=1}^{N_b} \|f_{\theta}(x_{b,i}) - y_{b,i}\|_2^2 \right] + \lambda_1 \|\theta\|_1 + \lambda_2 \|\theta\|_2^2 \quad (2.3)$$

$$+ \eta \sum_{j=1}^J \max(0, \tau_j - q_j(\theta))^2 + \mu \sum_{r=1}^R |c_r(\theta) - \hat{c}_r|. \quad (2.4)$$

A [equation \(2.3\)](#) mostra uma função objetivo com termo empírico, regularização, penalidades de restrição e distância a metas operacionais. Esse tipo de bloco é comum em capítulos de otimização, modelagem de desempenho e aprendizado de máquina.

2.4 Bloco técnico com equação

Regra editorial para matemática

Use equações numeradas quando elas forem citadas, discutidas ou reutilizadas. Use equações não numeradas apenas para derivações locais ou exemplos transitórios. Evite transformar texto explicativo em sequência de símbolos quando o argumento puder ser expresso de forma mais clara em linguagem natural.

Figuras, Gráficos e Blocos Multicoluna

Este capítulo reúne exemplos visuais usados com frequência em materiais técnicos: gráficos individuais, painéis de gráficos, fotografia centralizada, fotografia acompanhada de texto e blocos editoriais em duas, três e quatro colunas.

3.1 Um gráfico

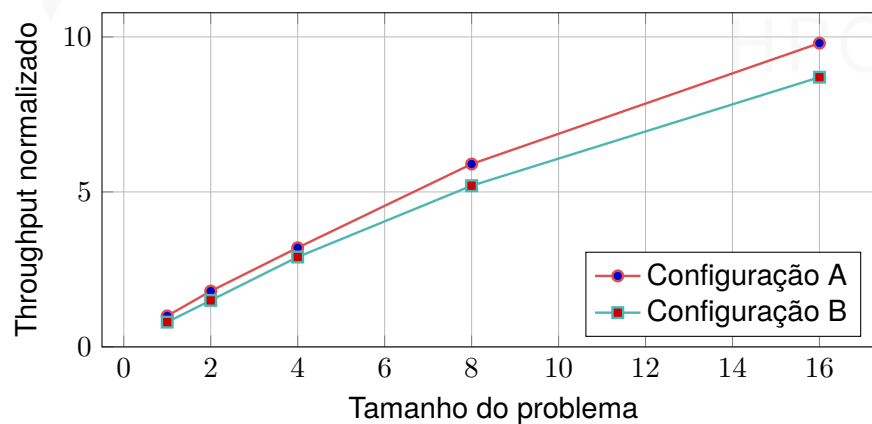
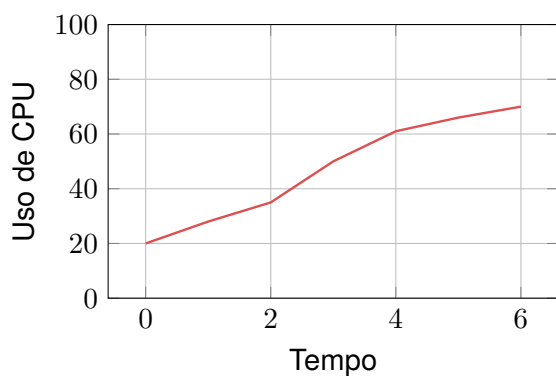
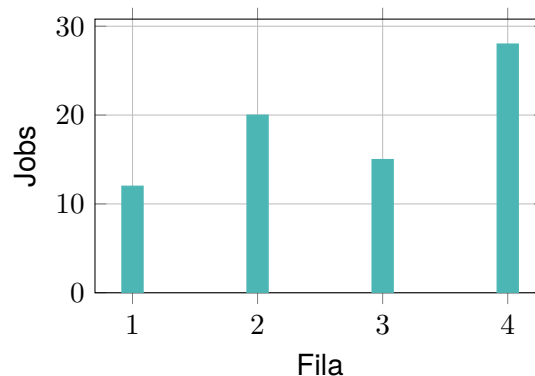


Figura 3.1: Exemplo de gráfico individual gerado em TikZ/PGFPlots

3.2 Dois gráficos lado a lado



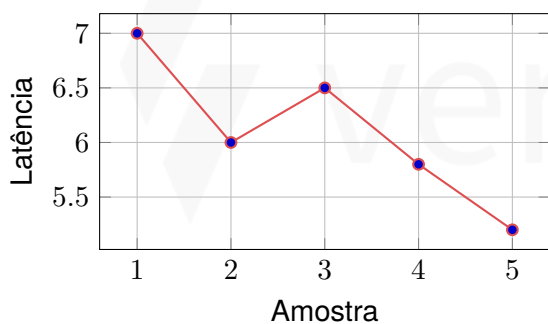
(a) Série temporal



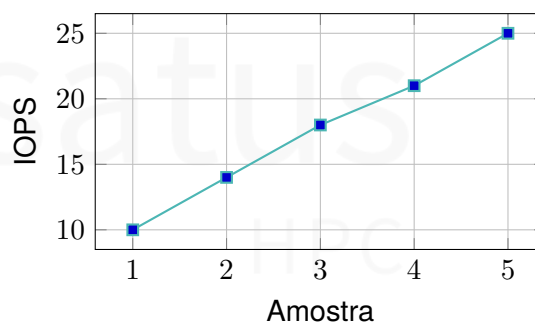
(b) Distribuição discreta

Figura 3.2: Exemplo de dois gráficos lado a lado

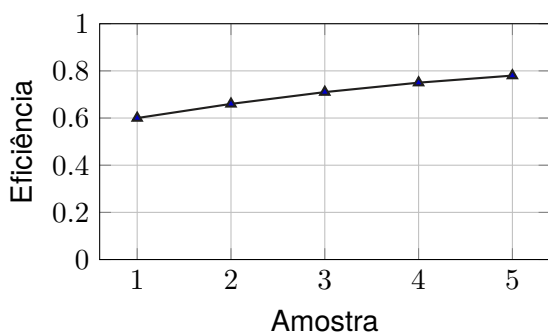
3.3 Quatro gráficos em matriz 2 x 2



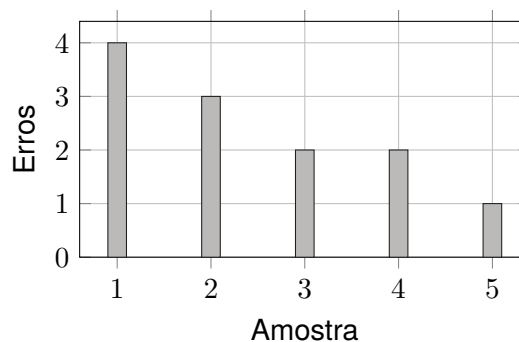
(a) Latência



(b) IOPS



(c) Eficiência



(d) Erros

Figura 3.3: Exemplo de quatro gráficos organizados em duas linhas e duas colunas

3.4 Foto centralizada

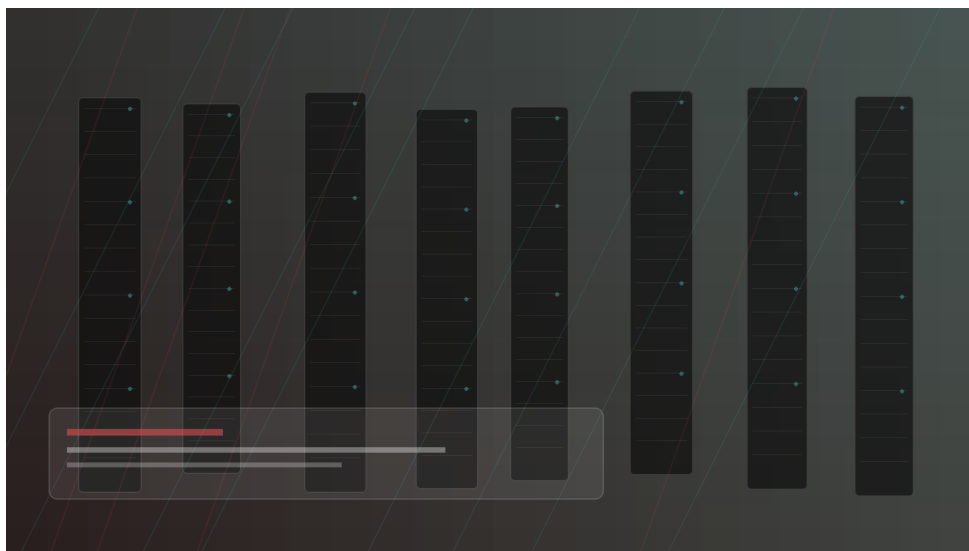
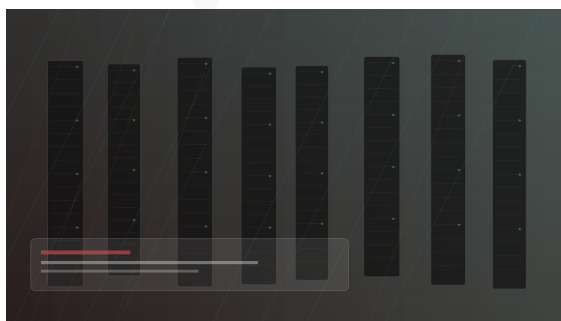


Figura 3.4: Exemplo de fotografia centralizada. Substitua o arquivo por uma fotografia real do produto, laboratório, cluster ou instalação.

3.5 Foto com texto ao lado



Uso recomendado. Esta composição é adequada para explicar uma imagem sem deslocar o leitor para outra página. Ela funciona bem em manuais, livros de laboratório, lâminas técnicas e relatórios de validação.

Critério editorial. O texto lateral deve interpretar a imagem, não repetir a legenda. Para fotografias de produto, descreva o contexto, o componente visível, o papel operacional e a evidência técnica.

Figura 3.5: Exemplo de fotografia acompanhada de texto lateral

3.6 Dois blocos em duas colunas

Bloco 1

Use duas colunas quando houver comparação direta entre conceitos, decisões arquiteturais, variantes de produto ou modos de operação.

Bloco 2

Evite colunas longas demais. Se o conteúdo exigir mais de meia página, prefira uma seção convencional com subtítulos.

3.7 Três blocos em três colunas

Entrada

Defina pré-condições, arquivos, parâmetros e dependências externas.

Processo

Liste os passos principais e destaque comandos ou decisões manuais.

Saída

Registre artefatos, logs, métricas, relatórios e critérios de aceite.

3.8 Quatro blocos em quatro colunas

Contexto

Descreva o problema técnico.

Decisão

Declare a escolha adotada.

Evidência

Aponte teste, métrica ou fonte.

Risco

Registre limitações e mitigação.



CAPÍTULO 4

Blocos de Código e Terminal

Este capítulo apresenta exemplos de código em dois regimes editoriais: blocos escuros, adequados para simular uma sessão de terminal ou uma janela de execução, e blocos claros, adequados para leitura de código-fonte em materiais didáticos. O template usa `listings` com `tcolorbox`, evitando `minted` para preservar portabilidade sem `shell-escape`.

4.1 Bloco pequeno de código em terminal

Terminal pequeno

```
1 def normalize(value, scale):
2     if scale == 0:
3         return 0.0
4     return value / scale
```

4.2 Bloco médio de código em terminal

Terminal médio

```
1 def summarize_jobs(jobs):
2     total_gpus = 0
3     total_nodes = 0
4     failed_jobs = []
5
6     for job in jobs:
7         total_gpus += job.get("gpus", 0)
8         total_nodes += job.get("nodes", 0)
9         if job.get("state") == "FAILED":
10             failed_jobs.append(job.get("id"))
11
12     return {
13         "jobs": len(jobs),
14         "gpus": total_gpus,
15         "nodes": total_nodes,
16         "failed": failed_jobs,
17     }
```

4.3 Bloco grande de código em terminal

Terminal grande

```
1 class ClusterPolicy:
2     def __init__(self, max_gpus_per_user, reserved_nodes):
3         self.max_gpus_per_user = max_gpus_per_user
4         self.reserved_nodes = set(reserved_nodes)
5
6     def can_schedule(self, request, usage):
7         user = request["user"]
8         required_gpus = request.get("gpus", 0)
9         candidate_nodes = request.get("nodes", [])
10
11         if usage.get(user, 0) + required_gpus > self.max_gpus_per_user:
12             return False, "gpu quota exceeded"
13
14         for node in candidate_nodes:
15             if node in self.reserved_nodes and not request.get("admin", False):
16                 return False, "reserved node requested"
17
18         if not self._has_required_topology(candidate_nodes):
19             return False, "topology constraint not satisfied"
20
21         return True, "accepted"
22
23     def _has_required_topology(self, nodes):
24         if len(nodes) <= 1:
25             return True
26         racks = {node.split("-")[0] for node in nodes}
27         return len(racks) <= 2
28
29 policy = ClusterPolicy(max_gpus_per_user=32, reserved_nodes=["rack9-node01"])
30 request = {
31     "user": "researcher",
32     "gpus": 8,
33     "nodes": ["rack1-node03", "rack1-node04"],
34     "admin": False,
35 }
36 usage = {"researcher": 16}
37 accepted, reason = policy.can_schedule(request, usage)
38 print(accepted, reason)
```

4.4 Bloco pequeno de código claro

Código pequeno

```
1 def bandwidth(bytes_written, seconds):  
2     return bytes_written / seconds
```

4.5 Bloco médio de código claro

Código médio

```
1 def moving_average(samples, window):  
2     if window <= 0:  
3         raise ValueError("window must be positive")  
4  
5     result = []  
6     for index in range(len(samples)):  
7         start = max(0, index - window + 1)  
8         segment = samples[start:index + 1]  
9         result.append(sum(segment) / len(segment))  
10  
11     return result
```

4.6 Bloco grande de código claro

Código grande

```
1 from dataclasses import dataclass
2
3 @dataclass
4 class Measurement:
5     timestamp: float
6     node: str
7     metric: str
8     value: float
9
10 class TelemetryBuffer:
11     def __init__(self, limit):
12         self.limit = limit
13         self.samples = []
14
15     def append(self, measurement):
16         self.samples.append(measurement)
17         if len(self.samples) > self.limit:
18             self.samples = self.samples[-self.limit:]
19
20     def select(self, metric, node=None):
21         selected = []
22         for sample in self.samples:
23             if sample.metric != metric:
24                 continue
25             if node is not None and sample.node != node:
26                 continue
27             selected.append(sample)
28         return selected
29
30     def aggregate_mean(self, metric, node=None):
31         selected = self.select(metric, node=node)
32         if not selected:
33             return None
34         return sum(sample.value for sample in selected) / len(selected)
35
36 buffer = TelemetryBuffer(limit=10000)
37 buffer.append(Measurement(0.0, "node001", "gpu_util", 71.0))
38 buffer.append(Measurement(1.0, "node001", "gpu_util", 78.0))
39 buffer.append(Measurement(2.0, "node001", "gpu_util", 82.0))
40 print(buffer.aggregate_mean("gpu_util", node="node001"))
```

Teoremas, Provas e Blocos de Destaque

Este capítulo valida a composição de enunciados matemáticos, demonstrações e blocos editoriais de ênfase. Esses elementos são importantes para livros de formação, materiais científicos, notas técnicas e documentação de arquitetura que contenha invariantes, propriedades formais ou regras normativas.

5.1 Enunciação de teorema matemático

Teorema 5.1 (Monotonicidade de custo acumulado) *Seja $c_i \geq 0$ o custo incremental de uma sequência finita de operações técnicas. Defina*

$$C_n = \sum_{i=1}^n c_i.$$

Então a sequência $(C_n)_{n \geq 1}$ é monotonicamente não decrescente.

5.2 Prova de teorema matemático

Demonstração. Considere dois índices consecutivos n e $n + 1$. Pela definição de custo acumulado,

$$C_{n+1} = \sum_{i=1}^{n+1} c_i = \sum_{i=1}^n c_i + c_{n+1} = C_n + c_{n+1}.$$

Como $c_{n+1} \geq 0$, segue que $C_{n+1} \geq C_n$. Portanto, a sequência de custos acumulados é monotonicamente não decrescente. $\square$

5.3 Enunciação de lema matemático

Lema 5.2 (Limite superior por capacidade) *Se cada tarefa consome no máximo $g_{\max}$ aceleradores e há N tarefas concorrentes, então o consumo total G satisfaz*

$$G \leq N g_{\max}.$$

O Lema 5.2 é um exemplo simples de limite usado em raciocínios de capacidade. Em um livro real, esse tipo de lema pode preparar uma proposição mais forte sobre escalabilidade, contenção de recursos ou escalonamento.

5.4 Enunciação de corolário matemático

Corolário 5.3 (Quota suficiente) *Se a quota disponível Q satisfaz $Q \geq N g_{\max}$, então a quota é suficiente para acomodar qualquer conjunto de N tarefas que obedeça ao limite individual $g_{\max}$.*

O Corolário 5.3 segue diretamente do Lema 5.2. O corolário registra uma consequência operacional simples: uma política de quota pode ser dimensionada a partir de um limite superior conservador.

5.5 Destaque importante de texto

Destaque importante

Use este bloco para uma afirmação normativa, uma restrição de projeto, uma decisão irreversível ou uma instrução que não deve ser confundida com observação lateral. Ele deve ser usado com parcimônia.

5.6 Destaque importante em bloco cinza claro

Destaque importante

Use este bloco quando o destaque for relevante, mas não crítico. Ele é adequado para recomendações editoriais, notas de implementação, critérios de revisão e advertências de baixa severidade.

5.7 Destaque importante em bloco colorido

Destaque importante

Use este bloco quando a variante colorida do documento puder reforçar visualmente um ponto de atenção. Nas variantes monocromáticas, o mesmo ambiente preserva legibilidade e passa automaticamente para uma composição sem cor.

